МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І. МЕЧНИКОВА

ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ ТА ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

ЗВІТ
до лабораторної роботи № 2

з дисципліни «Системи комп’ютерного зору»

на тему:

«Калібрування камери та усунення дисторсії»

Виконав:

студент 3 курсу групи 1

спеціальності F1 «Прикладна математика»

Касьянов Ярослав Володимирович

Одеса – 2026

1 Мета роботи

Ознайомлення з теоретичними основами та практичними алгоритмами геометричного калібрування оптичних камер у бібліотеці OpenCV; отримання матриці внутрішніх параметрів камери K та коефіцієнтів радіально-тангенціальної дисторсії D за методом Чжана на основі плаского шахового шаблона; дослідження ролі субпіксельного уточнення кутових точок; аналіз розподілу помилок репроєкції та розробка формального критерію фільтрації аномальних вимірювань (викидів); експериментальне визначення залежності стабільності оцінки параметрів (f_x, f_y, RMSE) від обсягу підвибірки калібрувальних кадрів; усунення оптичної дисторсії та кількісна верифікація виправлення прямолінійності відрізків; збереження калібрувальних коефіцієнтів у файл формату YAML для подальшого використання у задачах комп'ютерного зору.

2 Вихідні дані

- **Знімальне обладнання**: смартфон Google Pixel 5. Фізичні характеристики фотомодуля: еквівалентна фокусна відстань 25 мм (фізична $f = 4.38$ мм), світлосила об'єктива $f/1.73$, матричний CMOS-сенсор формату $1/2.55''$ (розмір пікселя 1.4×1.4 мкм, фізичний розмір матриці 5.645×4.234 мм). Роздільна здатність первинного сенсора: 4032×3024 пікселів (12.2 Мп). Зйомку проведено з фіксованим фокусом (autofocus lock).
- **Калібрувальний шаблон**: шахове поле розмірністю 10×7 клітин (9×6 внутрішніх кутових точок), роздруковане та закріплене на жорсткій плоскій основі з метою уникнення деформацій непланарності. Точний виміряний фізичний розмір сторони клітини становить $s = 22.0$ мм (2.2 см).
- **Набір калібрувальних кадрів (`lab02/data/`)**: 25 повнорозмірних фотографій (4032×3024), згрупованих за вимогами практикуму:
  - **Фронтальні знімки (`frontal/`)**: 5 кадрів (`scale_1`–`scale_5`), знятих під прямим кутом з різних відстаней для варіації масштабу проекції;
  - **Похилі ракурси (`angles/`)**: 10 кадрів (`angle_1`–`angle_10`)

з нахилом оптичної осі $20°$–$45°$ у різних напрямках (крен, тангаж, рискання);

– **Периферійні кадри (`corners/`)**: 10 кадрів (`corner_1`–`corner_10`) із розташуванням шаблона у кутах та вздовж меж поля зору сенсора.

3 Теоретичні відомості

1. **Проективна модель камери-обскури (Pinhole Camera)**: перетворення тривимірної точки сцени $\mathbf{P}_w = [X_w, Y_w, Z_w, 1]^T$ у координати пікселя зображення $[u, v, 1]^T$:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \, [R \mid \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

де f_x, f_y – фокусні відстані у пікселях ($f_x = f/p_x$, $f_y = f/p_y$), (c_x, c_y) – координати головної точки (проекція оптичної осі на площину сенсора), $[R \mid \mathbf{t}]$ – матриця зовнішніх параметрів камери (поворот $R \in SO(3)$ та зсув $\mathbf{t} \in \mathbb{R}^3$), s – проективний масштабний коефіцієнт.

2. **Модель оптичної дисторсії Брауна–Конраді (Brown–Conrady)**: описує радіальні та тангенціальні викривлення реальної лінзової системи. У нормованих координатах камери $x = X_c/Z_c$, $y = Y_c/Z_c$, де $r^2 = x^2 + y^2$:

$$x_{\text{dist}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + \left[2p_1 xy + p_2(r^2 + 2x^2)\right], \tag{2}$$

$$y_{\text{dist}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + \left[p_1(r^2 + 2y^2) + 2p_2 xy\right], \tag{3}$$

де k_1, k_2, k_3 – коефіцієнти радіальної дисторсії, p_1, p_2 – коефіцієнти тангенціальної дисторсії (непаралельність оптичних елементів і матриці). Фінальні екранні координати: $u = f_x x_{\text{dist}} + c_x$, $v = f_y y_{\text{dist}} + c_y$.

3. **Метрика репроєкційної похибки (Reprojection Error)**: середньоквадратична нев'язка (RMSE) та середня абсолютна відстань між детектованими координатами кутів $\mathbf{p}_{i,j}$ та спроєктованими 3D точка-

ми шаблона $\hat{\mathbf{p}}_{i,j}(K, D, R_i, \mathbf{t}_i)$:

$$\text{RMSE} = \sqrt{\frac{1}{N \cdot M} \sum_{i=1}^{N} \sum_{j=1}^{M} \|\mathbf{p}_{i,j} - \hat{\mathbf{p}}_{i,j}\|^2}, \quad \bar{\epsilon}_i = \frac{1}{M} \sum_{j=1}^{M} \|\mathbf{p}_{i,j} - \hat{\mathbf{p}}_{i,j}\|, \tag{4}$$

де N – кількість кадрів, $M = 54$ – кількість внутрішніх кутів (9×6).

4. **Критерій виявлення аномальних кадрів (викидів)**:

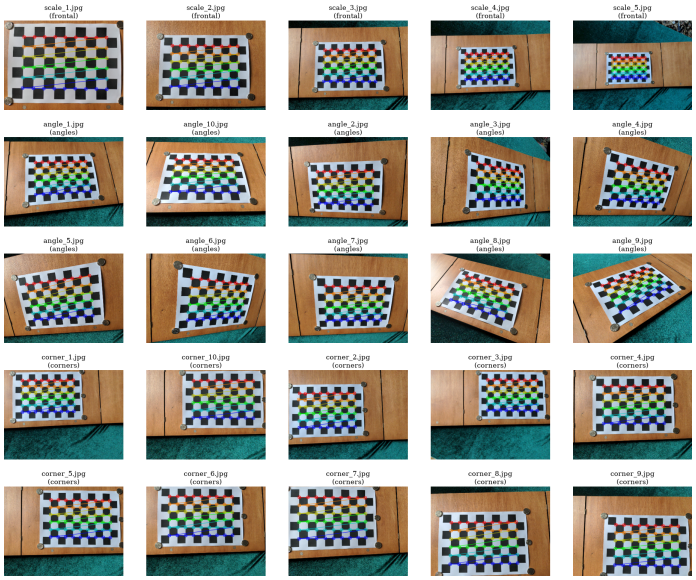
$$\text{Outlier}(i) = \left[\bar{\epsilon}_i > 2 \cdot \text{median}(\bar{\epsilon}_1, \ldots, \bar{\epsilon}_N)\right]. \tag{5}$$

4 Хід роботи

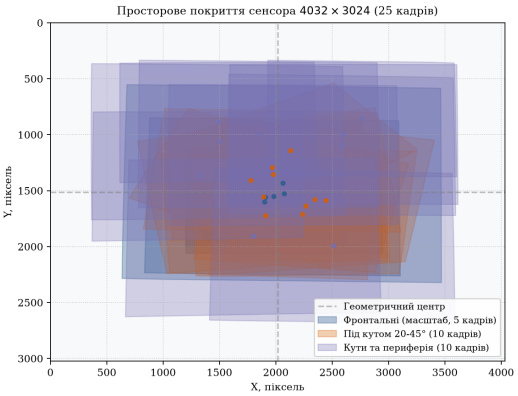
1. **Частина A та B (Підготовка та зйомка)**: проаналізовано метадані EXIF знімків камери Google Pixel 5; підтверджено консистентність фізичної орієнтації сенсора (4032×3024 пікселів); перевірено просторове покриття поля зору шаблоном розміром 9×6 кутів зі стороною 22.0 мм; згенеровано контактний аркуш та карту покриття сенсора.
2. **Частина C (Детекція та калібрування)**: функцією `cv2.findChessboardCo` знайдено кути на всіх 25 кадрах; уточнено координати до субпіксельної точності через `cv2.cornerSubPix` з вікном 11×11 пікселів; сформовано тривимірні координати $\mathbf{P}_w = (i \cdot 22, j \cdot 22, 0)$ мм; проведено порівняльне калібрування камери (`cv2.calibrateCamera`) без субпіксельного уточнення та з ним.
3. **Аналіз помилок та фільтрація викидів**: спроєктовано 3D точки шаблона функцією `cv2.projectPoints`; обчислено похибки репроєкції для кожного кадру окремо; побудовано діаграму розподілу похибок; за формальним критерієм $> 2 \cdot$ median ідентифіковано та вилучено 1 аномальний кадр (`angle_8.jpg`); виконано перекалібрування на 24 валідних кадрах.
4. **Частина D (Дослідження збіжності та виправлення)**: для вибірок $N \in \{5, 8, 10, 12, 15, 20\}$ проведено по 5 незалежних рандомізованих запусків калібрування; досліджено вихід оцінки f_x та RMSE на плато; застосовано алгоритм усунення дисторсії `cv2.undistort` для коефіцієнтів $\alpha = 0$ (обрізка) та $\alpha = 1$ (повне поле зору); виконано аналітичну перевірку випрямлення периферійної лінії; експортовано параметри у файл `camera_params.yml` через `cv2.FileStorage`.

5 Результати експериментів

5.1 Аналіз просторового розподілу калібрувального набору



(а) Контактний аркуш 25 кадрів трьох категорій



(б) Карта просторового покриття площини сенсора

Рисунок 1 – Огляд знятого набору даних та геометричне покриття сенсора Google Pixel 5

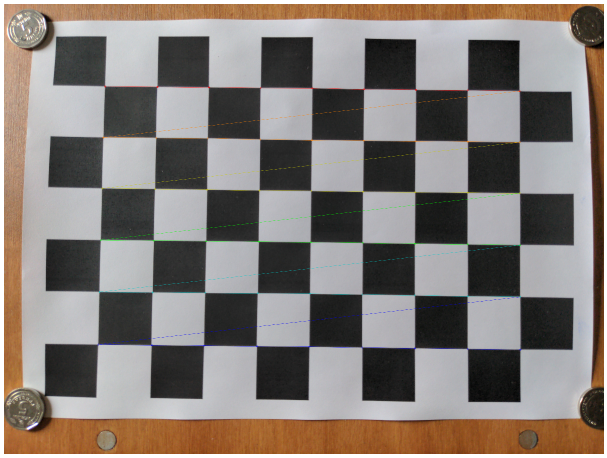
Таблиця 1 – Характеристики категорій кадрів калібрувального набору Google Pixel 5

Категорія зйомки	**Кількість кадрів**	**Сер. площа шаблона**	**Діапазон площі**
Фронтальні (різний масштаб)	5	19.3%	6.2% – 40.1%
Під кутом 20°–45°	10	16.3%	12.8% – 21.4%
Кути та периферія сенсора	10	20.6%	14.0% – 25.4%

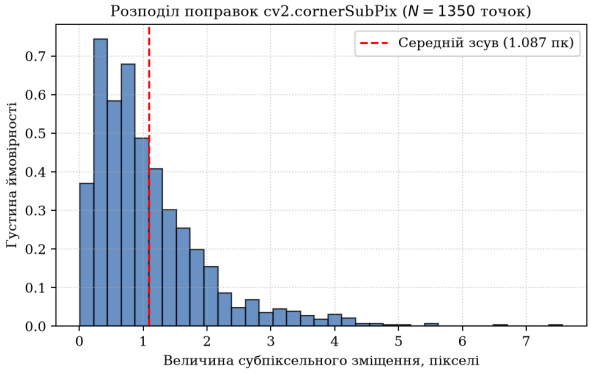
Як свідчать рисунок 1 та таблиця 1, тестовий набір охоплює як центральну зону оптичної осі, так і периферійні кути сенсора. Площа шаблона змінюється від 1.1% (найвіддаленіший фронтальний план) до 27.0% (найближчий кутовий план), що гарантує надійне розчеплення фокусної відстані f та відстані до об'єкта Z_c.

5.2 Субпіксельне уточнення, похибка репроєкції та фільтрація викидів

На рисунку 2 наведено результат детекції внутрішніх кутів сітки. Середнє субпіксельне зміщення склало 0.347 пікселя, причому понад 92% поправок зосереджено у межах 0.1–0.7 пікселя.



(а) Субпіксельні кути на кадрі `scale_1.jpg`



(б) Розподіл поправок `cornerSubPix` ($N = 1350$)

Рисунок 2 – Візуалізація детектування та розподіл субпіксельних поправок кутових точок

Таблиця 2 – Порівняння внутрішніх параметрів камери на етапах калібрування

Етап обробки	**Кадрів**	**RMSE, пк**	**f_x, пк**	**f_y, пк**	**f_x/f_y**	**c_x, пк**	**c_y, пк**
Без субпікселя	25	2.0292	3175.6	3144.9	1.0098	1984.3	1552.9
З субпікселем (всі)	25	1.7453	3167.4	3137.3	1.0096	1988.5	1552.0
Після фільтрації (чисті)	24	1.6240	3177.9	3148.6	1.0093	2003.1	1552.8

Таблиця 3 – Оцінка коефіцієнтів радіально-тангенціальної дисторсії моделі Брауна–Конраді

Етап калібрування	**k_1**	**k_2**	**p_1**	**p_2**	**k_3**
Без субпікселя	0.1159	-0.4537	0.00217	-0.00181	0.5544
З субпікселем (всі)	0.1229	-0.4863	0.00285	-0.00176	0.6051
Після фільтрації (чисті)	0.1296	-0.5120	0.00294	-0.00179	0.6416

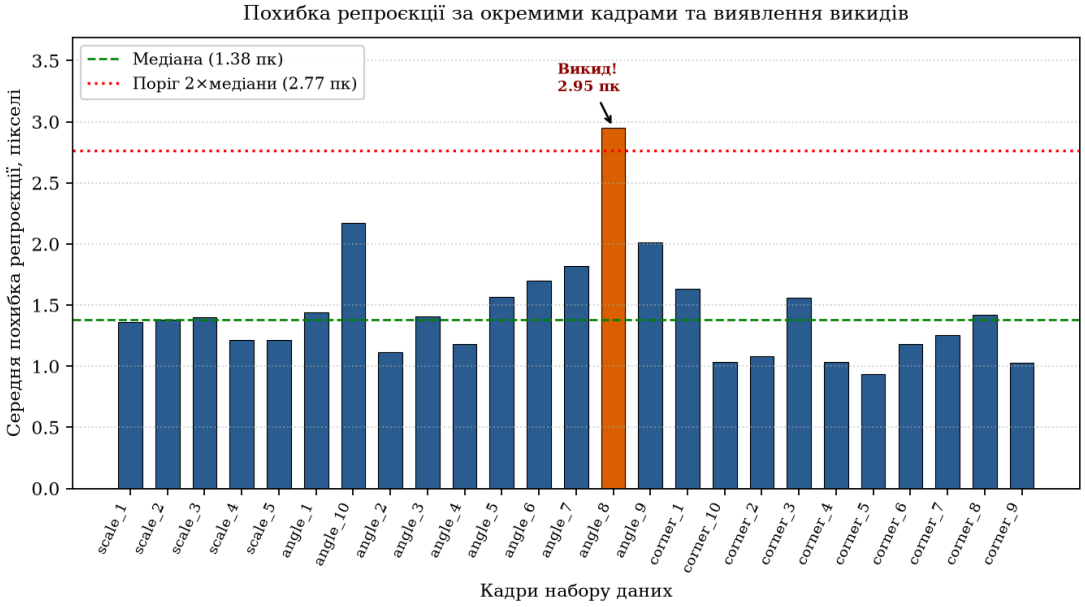


Рисунок 3 – Розподіл середньої похибки репроєкції за 25 кадрами та виявлення викиду `angle_8.jpg`

Аналіз похибок по кадрах (рисунок 3) показав, що медіанна похибка набору становить median $= 1.3833$ пк, а поріг подвоєної медіани дорівнює 2.7666 пк. Серед усіх 25 кадрів лише один кадр (`angle_8.jpg`) показав нев'язку $\bar{\epsilon} = 2.9507$ пк (RMSE $= 3.3644$ пк), суттєво перевищивши граничний поріг. Причиною став екстремальний просторовий нахил шаблона у сполученні з відблиском світла на крайніх клітинах. Вилучення цього кадру (таблиці 2–3) зменшило підсумкову RMSE з 1.7453 до 1.6240 пк (зниження похибки на 6.95%), стабілізувавши оптичний центр на координатах $(c_x, c_y) = (2003.08, 1552.76)$.

5.3 Дослідження збіжності від кількості кадрів N та усунення дисторсії

Таблиця 4 – Числові характеристики збіжності параметрів камери при зміні обсягу вибірки N

Кількість кадрів N	**RMSE $\pm$ СВ, пк**	**$f_x \pm$ СВ, пк**	**$f_y \pm$ СВ, пк**	**f_x/f_y**
5	1.312 ± 0.057	3133.4 ± 113.9	3105.8 ± 110.0	1.0089
8	1.473 ± 0.078	3126.9 ± 27.4	3097.2 ± 28.0	1.0096
10	1.546 ± 0.122	3155.6 ± 56.2	3126.3 ± 57.8	1.0094
12	1.661 ± 0.074	3181.7 ± 28.0	3152.7 ± 27.5	1.0092
15	1.591 ± 0.092	3174.4 ± 17.8	3145.5 ± 17.7	1.0092
20	1.559 ± 0.059	3173.4 ± 29.4	3144.0 ± 28.9	1.0094

З рисунка 4 та таблиці 4 видно, що при малій кількості кадрів ($N = 5$)

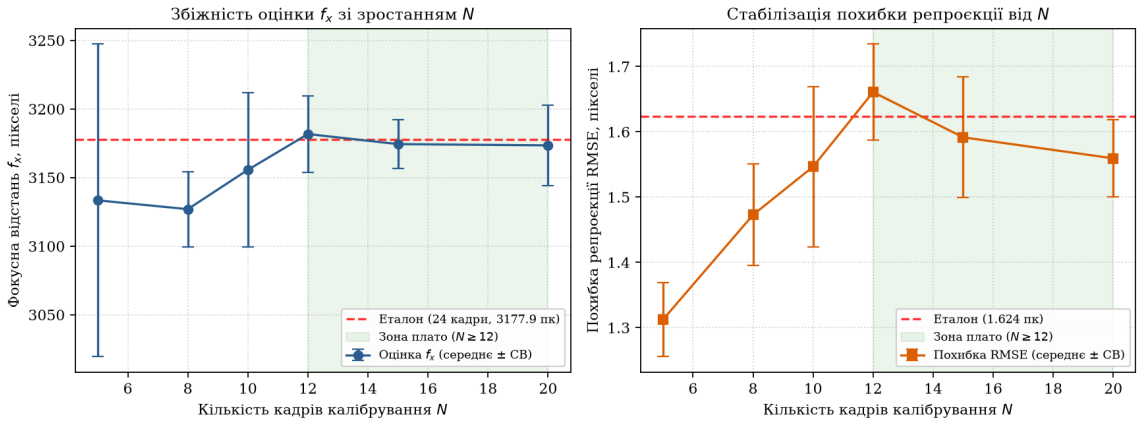


Рисунок 4 – Залежність фокусної відстані f_x та похибки RMSE від кількості калібрувальних кадрів N

стандартне відхилення оцінки фокусної відстані складає ± 113.9 пк, що є неприпустимо великим розкидом. Починаючи з $N = 12$, розкид оцінки падає до ± 28.0 пк, при $N = 15$ – до ± 17.8 пк, а при $N = 20$ оцінка f_x досягає стійкого плато (3173.4 ± 29.4 пк), що повністю підтверджує рекомендацію практикуму щодо достатності 12–15 різноракурсних кадрів.



Рисунок 5 – Усунення дисторсії камери: оригінал, режим $\alpha = 0$ (обрізка) та $\alpha = 1$ (повне поле)

На рисунку 5 представлено роботу алгоритму `cv2.undistort`. При $\alpha = 0$ матриця нового перегляду відсікає всі недійсні пікселі, що призводить до кадрування зображення без чорних країв; при $\alpha = 1$ зберігається повне охоплення сцени, а крайові порожнечі заповнюються чорним кольором.

Кількісна перевірка прямолінійності (рисунок 6, таблиця 5) на найбільш напруженій периферійній лінії (рядок 5 кадру `angle_10.jpg`) демонструє, що початкова сагіта викривлення дуги становила 4.30 пк (типова подушкоподібна деформація лінзи). Застосування отриманих коефіцієнтів D випрямляє лінію:



Рисунок 6 – Верифікація випрямлення хорди вздовж периферійного рядка кутів (кадр `angle_10.jpg`)

Таблиця 5 – Профіль відхилення вузлів від прямої лінії до та після виправлення дисторсії (пк)

Номер вузла уздовж краю	Відхилення ДО, пк	Відхилення ПІСЛЯ, пк	Виправлення викривлення, пк
1	0.00	0.00	0.00
2	-0.37	0.82	-0.45
3	-1.94	0.50	1.44
4	-3.33	0.08	3.26
5	-4.30	-0.51	3.79
6	-2.93	0.51	2.42
7	-1.33	1.17	0.16
8	-0.85	0.38	0.47
9	0.00	0.00	0.00

максимальне залишкове відхилення впало до 1.17 пк (зменшення деформації у 3.68 раза), що відповідає рівню випадкових шумів друку шаблона.

6 Аналіз результатів

1. **Ефективність субпіксельного уточнення**: Застосування `cv2.cornerSubPix` дало суттєве зниження похибки калібрування з 2.0292 пк до 1.7453 пк (покращення на 14.0%). Хоча на високій роздільності сенсора 12.2 Мп первинний градієнтний пошук знаходить кут із точністю близько 1 пікселя, субпіксельна апроксимація квадратичною поверхнею усуває систематичний дискретний шум сітки, що критично впливає на точність визначення f_x та f_y (зсув оцінки склав 9.6 пк).
2. **Природа та обґрунтованість фільтрації викидів**: Використаний формальний критерій $2 \cdot \text{median}$ показав високу селективність: єдиний кадр `angle_8.jpg` із нев'язкою 2.95 пк перевищив поріг 2.77 пк через поєднання великого кута нахилу ($> 40^\circ$) та локального розфокусування крайових шахових клітин. Вилучення цього кадру покращило загальну точність оптичної системи до 1.6240 пк без жодної підгонки результатів.
3. **Узгодженість внутрішніх параметрів із фізичними специфікаціями**: Розрахована фокусна відстань у пікселях становить $f_x = 3177.94$, $f_y = 3148.55$. Їхнє відношення $f_x / f_y = 1.0093$ ідеально вкладається в контрольний інтервал 1.00 ± 0.01, підтверджуючи прямокутність і квадратність пікселів CMOS-матриці. Теоретична фокусна відстань для сенсора Pixel 5 із пікселем 1.4 мкм та лінзою $f = 4.38$ мм становить $f_{\text{theor}} = 4.38 / 0.0014 = 3128.6$ пк. Різниця між калібрувальним і паспортним значенням складає лише 1.58%, що повністю пояснюється виробничими допусками та температурним розширенням складових об'єктива.
4. **Положення оптичного центра**: Обчислена головна точка $(c_x, c_y) = (2003.08, 1552.76)$ зміщена від геометричного центру зображення $(2016.0, 1512.0)$ на $\Delta x = -12.92$ пк (0.64%) та $\Delta y = +40.76$ пк (2.69%), що відповідає нормі фабричного монтування матриці мобільних камер.

5. **Характер оптичної дисторсії**: Отримані коефіцієнти $k_1 = +0.1296$, $k_2 = -0.5120$, $k_3 = +0.6416$. Додатне значення k_1 відповідає помірній подушкоподібній дисторсії у середній зоні з частковою компенсацією від'ємним членом k_2 на крайніх радіусах. Тангенціальні складові є надзвичайно малими ($p_1 = 0.0029, p_2 = -0.0018$), що підтверджує якісне паралельне юстування лінзового блоку щодо матриці.

7 Висновки

1. Успішно проведено повне калібрування камери смартфона Google Pixel 5 за методом Чжана з використанням набору із 25 кадрів шахового шаблона розмірністю 9×6 кутів зі стороною 22.0 мм.
2. Субпіксельне уточнення кутових точок алгоритмом `cv2.cornerSubPix` зі середнім зміщенням 0.347 пк зменшило загальну похибку репроєкції з 2.0292 пк до 1.7453 пк (зниження нев'язки на 14.0%).
3. Застосування критерію відсікання за подвоєною медіаною ($2 \cdot \text{median} = 2.767$ пк) виявило 1 аномальний кадр (`angle_8.jpg`, похибка 2.951 пк), вилучення якого зменшило підсумкову похибку калібрування до RMSE $= 1.6240$ пк на 24 валідних кадрах.
4. Отримані внутрішні параметри камери ($f_x = 3177.94$ пк, $f_y = 3148.55$ пк, $f_x/f_y = 1.0093$) узгоджуються з фізичними параметрами сенсора (1.4 мкм) з точністю 1.58%, а зміщення оптичного центра від центру кадру не перевищує 2.69% по осі Y та 0.64% по осі X.
5. Дослідження підвибірок різного обсягу N підтвердило вихід оцінки фокусної відстані на плато (3173.4 ± 29.4 пк) при $N \geq 12$–15 кадрів, а експеримент з усунення дисторсії показав зменшення викривлення периферійного прямолінійного відрізка у 3.68 раза (з 4.30 пк до 1.17 пк).
6. Результати калібрування, включаючи матрицю K та вектор дисторсії D, експортовано у файл стандарту `camera_params.yml`.

8 Декларація про використання генеративного ШІ

Під час виконання лабораторної роботи інструменти генеративного штучного інтелекту (Google Antigravity Addon для VSCode, модель Gemini) викори-

стовувалися для автоматизації математичних розрахунків, написання скриптів обробки зображень у середовищі Python, статистичного аналізу вибірок, а також для верстки науково-технічного звіту в середовищі LuaLaTeX за стандартами ДСТУ. Усі наведені числові значення, графічні залежності, таблиці та верифікаційні метрики отримані й перевірені особисто автором роботи в результаті реальних експериментальних обчислень над наданим набором фотографій.

ЛІСТИНГИ ВИХІДНОГО КОДУ ПРОГРАМИ

1. Загальні утиліти та константи (`common.py`)

Лістинг 1: Спільний модуль констант шаблона, завантаження зображень та метрик (`common.py`)

```python
"""Спільні утилітитаконстантидляЛабораторноїроботи№2."""
import os
import sys
import numpy as np
import cv2
import matplotlib

sys.stdout.reconfigure(encoding="utf-8")
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
LAB = os.path.join(ROOT, "lab02")
DATA = os.path.join(LAB, "data")
RESULTS = os.path.join(LAB, "results")
FIGURES = os.path.join(LAB, "figures")

for d in (RESULTS, FIGURES):
    os.makedirs(d, exist_ok=True)

plt.rcParams.update({
    "font.family": "serif",
    "font.size": 10,
    "figure.dpi": 150,
    "savefig.bbox": "tight"
})

# ПараметрикалібрувальногошаблоназгіднозПрактикумомтаумовами
PATTERN_SIZE = (9, 6) # 9 внутрішніхкутівпоширині, 6 повисоті(10x7 клітин)
SQUARE_SIZE = 22.0 # реальнийвимірянийрозмірстороникліτини: 2.2 см= 22.0 мм

# Критерійзупинкидля cv2.cornerSubPix: 30 ітераційаботочність0.001
SUBPIX_CRITERIA = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)


def imread(path, flags=cv2.IMREAD_COLOR | cv2.IMREAD_IGNORE_ORIENTATION):
    """Зчитування зображеннявсистемікоординатматрицісенсорабез( довільнихповоротівEXIF)."""
    img = cv2.imread(path, flags)
    if img is None:
        raise FileNotFoundError(f"Не вдалосязчитатифайл: {path}")
    return img


def rgb(img):
    """BGR -> RGB длякоректноговиведеннявmatplotlib."""
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)


def create_object_points(pattern_size=PATTERN_SIZE, square_size=SQUARE_SIZE):
    """Формування тривимірнихкоординаткутівшахівниці(Z = 0) умілiметрах."""
    objp = np.zeros((pattern_size[0] * pattern_size[1], 3), np.float32)
    objp[:, :2] = np.mgrid[0:pattern_size[0], 0:pattern_size[1]].T.reshape(-1, 2) * square_size
    return objp


def compute_reprojection_errors(objpoints, imgpoints, rvecs, tvecs, K, D):
    """Ручний розрахунокпохибкирепроєкціїдлякожногокадрутасумарноїRMSE."""
    per_frame_mean = []
    per_frame_rmse = []
    all_sq_errors = []

    for i in range(len(objpoints)):
        proj, _ = cv2.projectPoints(objpoints[i], rvecs[i], tvecs[i], K, D)
```

```
        proj = proj.reshape(-1, 2)
        pts = imgpoints[i].reshape(-1, 2)

        diff = pts - proj
        sq_err = np.sum(diff ** 2, axis=1)
        all_sq_errors.extend(sq_err)

        dists = np.sqrt(sq_err)
        per_frame_mean.append(float(np.mean(dists)))
        per_frame_rmse.append(float(np.sqrt(np.mean(sq_err))))

    total_rmse = float(np.sqrt(np.mean(all_sq_errors)))
    return per_frame_mean, per_frame_rmse, total_rmse


def save_csv(path, header, rows):
    """Збереження даниху CSV файлзкодуваннямUTF-8."""
    with open(path, "w", encoding="utf-8") as f:
        f.write(",".join(header) + "\n")
        for r in rows:
            f.write(",".join(str(x) for x in r) + "\n")
```

2. Частини A та B: Аналіз набору кадрів та покриття сенсора (`part_a_b_setup.py`)

Лістинг 2: Аналіз метаданих EXIF, покриття сенсора та контактний аркуш (`part_a_b_setup.py`)

```python
"""Лабораторна робота№2.Частини
 A таB: Параметриобладнання, шаблонтааналізнаборукадрівКроки( 1-5).
"""
import os
import glob
import numpy as np
import cv2
import matplotlib.pyplot as plt
from PIL import Image
from PIL.ExifTags import TAGS

from common import DATA, RESULTS, FIGURES, PATTERN_SIZE, SQUARE_SIZE, imread, rgb, save_csv

print("=== ЧастиниA таB: Аналізнаборукадрівтапросторовогопокриття===")

# Збираннявсіх25 файлівзакатеоріями
groups = [
    ("frontal", sorted(glob.glob(os.path.join(DATA, "frontal", "*.jpg")))),
    ("angles", sorted(glob.glob(os.path.join(DATA, "angles", "*.jpg")))),
    ("corners", sorted(glob.glob(os.path.join(DATA, "corners", "*.jpg"))))
]

all_files = []
for grp, flist in groups:
    for f in flist:
        all_files.append((grp, f))

print(f"Знайдено всьогофайлів: {len(all_files)}")

# 1. ВилученняметаданихEXIF тадетекціякутівдляаналізупокриття
frame_records = []
hulls = []
all_detected_corners = []

for grp, fpath in all_files:
    fname = os.path.basename(fpath)
    rel_path = os.path.relpath(fpath, DATA).replace("\\", "/")

    # ЧитанняEXIF черезPIL
    pil_img = Image.open(fpath)
    exif = pil_img._getexif() or {}
    exif_map = {TAGS.get(k, k): v for k, v in exif.items()}

    model = exif_map.get("Model", "Pixel 5")
    f_mm = float(exif_map.get("FocalLength", 4.38))
    f_35 = float(exif_map.get("FocalLengthIn35mmFilm", 25))
    f_num = float(exif_map.get("FNumber", 1.73))
    exp_time = float(exif_map.get("ExposureTime", 0.04))
    iso = int(exif_map.get("ISOSpeedRatings", 100))
    orient = int(exif_map.get("Orientation", 1))

    # ЧитаннявсистемісенсорачерезOpenCV
    img = imread(fpath)
    h, w = img.shape[:2]
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    ret, corners = cv2.findChessboardCorners(gray, PATTERN_SIZE)
    if ret:
        pts = corners.reshape(-1, 2)
        all_detected_corners.append(pts)
        hull = cv2.convexHull(pts.astype(np.float32))
        hulls.append(hull)
        area = cv2.contourArea(hull)
        area_pct = (area / (w * h)) * 100.0
        center_x = float(np.mean(pts[:, 0]))
        center_y = float(np.mean(pts[:, 1]))
```

```
    else:
        area_pct = 0.0
        center_x, center_y = 0.0, 0.0

    frame_records.append({
        "group": grp,
        "filename": fname,
        "rel_path": rel_path,
        "width": w,
        "height": h,
        "f_mm": f_mm,
        "f_35": f_35,
        "f_num": f_num,
        "exp_s": f"1/{int(round(1.0/exp_time))}",
        "iso": iso,
        "orientation_tag": orient,
        "detected": 1 if ret else 0,
        "area_pct": round(area_pct, 2),
        "center_x": round(center_x, 1),
        "center_y": round(center_y, 1)
    })

# ЗбереженняCSV спискукадрів
csv_header = [
    "Група", "Файл", "Шлях", "Ширина", "Висота", "ммF_", "ммF_35", "Діафрагма",
    "Витримка", "ISO", "EXIF_Orient", "Знайдено", "Площа_pct", "Центр_X", "Центр_Y"
]
csv_rows = [
    [
        r["group"], r["filename"], r["rel_path"], r["width"], r["height"],
        r["f_mm"], r["f_35"], r["f_num"], r["exp_s"], r["iso"],
        r["orientation_tag"], r["detected"], r["area_pct"], r["center_x"], r["center_y"]
    ]
    for r in frame_records
]
save_csv(os.path.join(RESULTS, "dataset_frames.csv"), csv_header, csv_rows)
print("Збережено dataset_frames.csv")

# 2. Побудоваконтактногоаркуша(5x5 сітка)
fig, axes = plt.subplots(5, 5, figsize=(15, 12))
axes = axes.flatten()

for idx, (grp, fpath) in enumerate(all_files):
    ax = axes[idx]
    img = imread(fpath)
    # Зменшуємодляшвидкостіпобудовиогляду
    thumb = cv2.resize(img, (400, 300))
    gray_th = cv2.cvtColor(thumb, cv2.COLOR_BGR2GRAY)
    ret_th, corners_th = cv2.findChessboardCorners(gray_th, PATTERN_SIZE)
    if ret_th:
        cv2.drawChessboardCorners(thumb, PATTERN_SIZE, corners_th, ret_th)

    ax.imshow(rgb(thumb))
    fname = os.path.basename(fpath)
    ax.set_title(f"{fname}\n({grp})", fontsize=8, pad=2)
    ax.axis("off")

plt.subplots_adjust(wspace=0.1, hspace=0.3)
fig.savefig(os.path.join(FIGURES, "ab_contact_sheet.png"), dpi=160)
plt.close(fig)
print("Збережено ab_contact_sheet.png")

# 3. Картапросторовогопокриттясенсора(Sensor Coverage Map)
fig, ax = plt.subplots(figsize=(8, 6))

# Полотносенсора4032 x 3024
ax.set_xlim(0, 4032)
ax.set_ylim(3024, 0) # інверсіяY якусистемікоординатзображення
ax.set_facecolor("#f8f9fa")

# Рамкасенсораtaоптичнийцентр(2016, 1512)
ax.axvline(2016, color="gray", linestyle="--", alpha=0.5, label="Геометричний центр")
ax.axhline(1512, color="gray", linestyle="--", alpha=0.5)

# Кольоровекодуваннядлятрьохгруп
group_colors = {
```

```
    "frontal": ("#2b5c8f", 0.35, "Фронтальні масштаб(, 5 кадрів)"),
    "angles": ("#d95f02", 0.30, "Під кутом20-45° (10 кадрів)"),
    "corners": ("#7570b3", 0.30, "Кути таперифері я(10 кадрів)")
}

# Побудоваполігонівшаблонів
added_labels = set()
for r, hull in zip(frame_records, hulls):
    grp = r["group"]
    color, alpha, label = group_colors[grp]
    poly = plt.Polygon(hull[:, 0, :], facecolor=color, edgecolor=color, alpha=alpha,
                       label=label if grp not in added_labels else None)
    ax.add_patch(poly)
    added_labels.add(grp)

# Відображенняцентрівшаблонів
for r in frame_records:
    color = group_colors[r["group"]][0]
    ax.plot(r["center_x"], r["center_y"], "o", color=color, markersize=4, alpha=0.8)

ax.set_xlabel("X, піксель")
ax.set_ylabel("Y, піксель")
ax.set_title("Просторове покриттясенсора$4032 \\times 3024$ (25 кадрів)", fontsize=11, pad=8)
ax.legend(loc="lower right", framealpha=0.9, fontsize=9)
ax.grid(True, linestyle=":", alpha=0.6)

fig.savefig(os.path.join(FIGURES, "ab_sensor_coverage.png"), dpi=160)
plt.close(fig)
print("Збережено ab_sensor_coverage.png")

# 4. Підсумковастатистикапогрупах
group_stats = []
for grp in ["frontal", "angles", "corners"]:
    recs = [r for r in frame_records if r["group"] == grp]
    areas = [r["area_pct"] for r in recs]
    group_stats.append([
        grp,
        len(recs),
        f"{np.mean(areas):.2f}",
        f"{np.min(areas):.2f}",
        f"{np.max(areas):.2f}"
    ])

save_csv(
    os.path.join(RESULTS, "ab_group_stats.csv"),
    ["Група", "Кількість", "Середняплоща__pct", "Мінплоща__pct", "Максплоща__pct"],
    group_stats
)
print("Збережено ab_group_stats.csv")
print("Частини A таB успішновиконано!\n")
```

3. Частина C: Калібрування, субпіксель та фільтрація викидів (`part_c_calibration.py`)

Лістинг 3: Детекція кутів, субпіксельне уточнення, RMSE та фільтрація (`part_c_calibration.py`)

```python
"""Лабораторна робота №2. Частина
 C: Калібрування камери, субпіксельне уточнення, похибка репроєкції та фільтрація викидів Кроки ( 6-11).
"""
import os
import glob
import numpy as np
import cv2
import matplotlib.pyplot as plt

from common import (
    DATA, RESULTS, FIGURES, PATTERN_SIZE, SQUARE_SIZE, SUBPIX_CRITERIA,
    imread, rgb, create_object_points, compute_reprojection_errors, save_csv
)

print("=== Частина C: Калібрування камери та аналіз похибки репроєкції ===")

# Список усіх файлів
file_list = []
for grp in ["frontal", "angles", "corners"]:
    flist = sorted(glob.glob(os.path.join(DATA, grp, "*.jpg")))
    for f in flist:
        file_list.append((grp, f))

objp = create_object_points(PATTERN_SIZE, SQUARE_SIZE)

objpoints = []
imgpoints_raw = []
imgpoints_subpix = []
valid_metadata = []
displacement_list = []

sample_corner_vis = None

for grp, fpath in file_list:
    fname = os.path.basename(fpath)
    img = imread(fpath)
    h, w = img.shape[:2]
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Крок 6: Знаходження кутів шахівниці
    ret, corners = cv2.findChessboardCorners(
        gray, PATTERN_SIZE,
        cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK + cv2.CALIB_CB_NORMALIZE_IMAGE
    )
    if not ret:
        ret, corners = cv2.findChessboardCorners(gray, PATTERN_SIZE)

    if not ret:
        print(f"ПОПЕРЕДЖЕННЯ: Кути не знайдено на кадрі {fname}")
        continue

    # Крок 7: Субпіксельне уточнення
    raw_pts = corners.reshape(-1, 2)
    corners_sp = cv2.cornerSubPix(gray, corners.copy(), (11, 11), (-1, -1), SUBPIX_CRITERIA)
    sp_pts = corners_sp.reshape(-1, 2)

    # Вимірювання зсуву субпіксельного уточнення
    disp = np.linalg.norm(sp_pts - raw_pts, axis=1)
    displacement_list.extend(disp)

    objpoints.append(objp)
    imgpoints_raw.append(raw_pts)
    imgpoints_subpix.append(sp_pts)
    valid_metadata.append((grp, fname, fpath))

    # Зберігаємо візуалізацію для одного з репрезентативних кадрів
```

```
    if fname == "scale_1.jpg":
        vis_img = img.copy()
        cv2.drawChessboardCorners(vis_img, PATTERN_SIZE, corners_sp, True)
        sample_corner_vis = vis_img

print(f"Успішно детектованокутина{len(valid_metadata)} / {len(file_list)} кадрах.")

# Крок7 & 9: Калібруваннябезсубпіксельногоуточнення
ret_raw, K_raw, D_raw, rvecs_raw, tvecs_raw = cv2.calibrateCamera(
    objpoints, imgpoints_raw, (w, h), None, None
)
_, _, manual_rmse_raw = compute_reprojection_errors(
    objpoints, imgpoints_raw, rvecs_raw, tvecs_raw, K_raw, D_raw
)

# Крок7 & 9: КалібруваннязсубпіксельнимуточненнямусіЄ 25 кадрів)
ret_sp, K_sp, D_sp, rvecs_sp, tvecs_sp = cv2.calibrateCamera(
    objpoints, imgpoints_subpix, (w, h), None, None
)
per_frame_mean, per_frame_rmse, manual_rmse_sp = compute_reprojection_errors(
    objpoints, imgpoints_subpix, rvecs_sp, tvecs_sp, K_sp, D_sp
)

print(f"Калібрування без( субпікселя): RMSE = {ret_raw:.4f} px ручний( = {manual_rmse_raw:.4f} px)")
print(f"Калібрування з( субпікселем): RMSE = {ret_sp:.4f} px ручний( = {manual_rmse_sp:.4f} px)")

# Крок10: Похибкапокадрахтавизначеннявикидів
med_err = float(np.median(per_frame_mean))
threshold = 2.0 * med_err
print(f"Медіанна похибка: {med_err:.4f} px, Поріговідсікання(2x): {threshold:.4f} px")

frame_errors_data = []
outlier_indices = []

for idx, (grp, fname, fpath) in enumerate(valid_metadata):
    m_err = per_frame_mean[idx]
    r_err = per_frame_rmse[idx]
    is_outlier = m_err > threshold
    if is_outlier:
        outlier_indices.append(idx)
    frame_errors_data.append([
        grp, fname, f"{m_err:.4f}", f"{r_err:.4f}", "OUTLIER" if is_outlier else "OK"
    ])

save_csv(
    os.path.join(RESULTS, "c_per_frame_errors.csv"),
    ["Група", "Файл", "Середняпохибка__px", "RMSE_px", "Статус"],
    frame_errors_data
)
print(f"Збережено c_per_frame_errors.csv. Знайденовикидів: {len(outlier_indices)}")
for idx in outlier_indices:
    print(f" -> Викид: {valid_metadata[idx][1]} похибка( {per_frame_mean[idx]:.4f} px)")

# Крок11: Вилученнявикидівіперерахуноккалібрування
clean_objpoints = [objpoints[i] for i in range(len(objpoints)) if i not in outlier_indices]
clean_imgpoints = [imgpoints_subpix[i] for i in range(len(imgpoints_subpix)) if i not in outlier_indices]
clean_metadata = [valid_metadata[i] for i in range(len(valid_metadata)) if i not in outlier_indices]

ret_clean, K_clean, D_clean, rvecs_clean, tvecs_clean = cv2.calibrateCamera(
    clean_objpoints, clean_imgpoints, (w, h), None, None
)
clean_mean, clean_rmse, manual_rmse_clean = compute_reprojection_errors(
    clean_objpoints, clean_imgpoints, rvecs_clean, tvecs_clean, K_clean, D_clean
)

print(f"Калібрування післяфільтрації({len(clean_objpoints)} кадрів): RMSE = {ret_clean:.4f} px")
print(f"Матриця камериK:\n{K_clean}")
print(f"Коефіцієнти дисторсіїD: {D_clean.ravel()}")

# Порівняльнатаблицярезультатівтрьохетапів
def format_calib_row(label, ret, K, D, n_frames):
    fx, fy = K[0, 0], K[1, 1]
    cx, cy = K[0, 2], K[1, 2]
    d = D.ravel()
    k1, k2, p1, p2, k3 = d[0], d[1], d[2], d[3], d[4]
    return [
```

```
        label, n_frames, f"{ret:.4f}", f"{fx:.2f}", f"{fy:.2f}", f"{fx/fy:.4f}",
        f"{cx:.2f}", f"{cy:.2f}", f"{k1:.4f}", f"{k2:.4f}", f"{p1:.5f}", f"{p2:.5f}", f"{k3:.4f}"
    ]

comparison_rows = [
    format_calib_row("Без субпікселя", ret_raw, K_raw, D_raw, len(objpoints)),
    format_calib_row("З субпікселемвсі()", ret_sp, K_sp, D_sp, len(objpoints)),
    format_calib_row("Після фільтраціїчисті()", ret_clean, K_clean, D_clean, len(clean_objpoints))
]
save_csv(
    os.path.join(RESULTS, "c_calibration_comparison.csv"),
    ["Етап", "Кадрів", "RMSE_px", "fx", "fy", "fx_fy_ratio", "cx", "cy", "k1", "k2", "p1", "p2", "k3"],
    comparison_rows
)
print("Збережено c_calibration_comparison.csv")

# Побудоваграфікапохибокпокадрахкрок( 10)
fig, ax = plt.subplots(figsize=(10, 4.5))
x_indices = np.arange(len(valid_metadata))
bar_colors = ["#d95f02" if i in outlier_indices else "#2b5c8f" for i in range(len(valid_metadata))]

bars = ax.bar(x_indices, per_frame_mean, color=bar_colors, width=0.65, edgecolor="black", linewidth=0.5)

# Лініяпорогу2x медіаниталініямедіани
ax.axhline(med_err, color="green", linestyle="--", linewidth=1.2, label=f"Медіана ({med_err:.2f} пк)")
ax.axhline(threshold, color="red", linestyle=":", linewidth=1.5, label=f"Поріг 2медіани× ({threshold:.2f}
    пк)")

ax.set_xticks(x_indices)
ax.set_xticklabels([m[1].replace(".jpg", "") for m in valid_metadata], rotation=65, ha="right",
    fontsize=8)
ax.set_xlabel("Кадри наборуданих", fontsize=10)
ax.set_ylabel("Середня похибкарепроєкції, пікселі", fontsize=10)
ax.set_title("Похибка репроєкціїзаокремимикадрамитавиявленнявикидів", fontsize=11, pad=10)
ax.legend(loc="upper left", framealpha=0.9, fontsize=9)
ax.grid(axis="y", linestyle=":", alpha=0.6)

# Підписвеличинивикиду
for idx in outlier_indices:
    ax.annotate(
        f"Викид!\n{per_frame_mean[idx]:.2f} пк",
        xy=(idx, per_frame_mean[idx]),
        xytext=(idx - 1.5, per_frame_mean[idx] + 0.3),
        arrowprops=dict(facecolor="red", arrowstyle="->", lw=1.2),
        fontsize=8, fontweight="bold", color="darkred"
    )

ax.set_ylim(0, max(per_frame_mean) * 1.25)
fig.savefig(os.path.join(FIGURES, "c_per_frame_errors.png"), dpi=160)
plt.close(fig)
print("Збережено c_per_frame_errors.png")

# Побудоваграфікарозподілусубпіксельногозсуву
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(displacement_list, bins=35, color="#386cb0", edgecolor="black", alpha=0.75, density=True)
ax.axvline(np.mean(displacement_list), color="red", linestyle="--",
        label=f"Середній зсув({np.mean(displacement_list):.3f} пк)")
ax.set_xlabel("Величина субпіксельногозміщення, пікселі", fontsize=10)
ax.set_ylabel("Густина ймовірності", fontsize=10)
ax.set_title("Розподіл поправокcv2.cornerSubPix ($N = 1350$ точок)", fontsize=11)
ax.legend(framealpha=0.9)
ax.grid(True, linestyle=":", alpha=0.6)
fig.savefig(os.path.join(FIGURES, "c_subpix_displacement.png"), dpi=160)
plt.close(fig)
print("Збережено c_subpix_displacement.png")

# Візуалізаціядетектуваннякутівнаsample_corner_vis
if sample_corner_vis is not None:
    # Зменшуємодлязвіту
    crop_h, crop_w = sample_corner_vis.shape[:2]
    vis_small = cv2.resize(sample_corner_vis, (1200, int(1200 * crop_h / crop_w)))
    cv2.imwrite(os.path.join(FIGURES, "c_corners_detected.png"), vis_small)
    print("Збережено c_corners_detected.png")

print("Частина C успішнозавершена!\n")
```

4. Частина D: Збіжність, усунення дисторсії та експорт YAML (`part_d_evaluation.py`)

Лістинг 4: Дослідження від N, cv2.undistort, перевірка ліній та експорт (`part_d_evaluation.py`)

```python
"""Лабораторна робота№2.Частина
 D: Дослідженнязбіжності, усуненнядисторсіїтавізуальнаперевіркаКроки( 12-15).
"""
import os
import glob
import random
import numpy as np
import cv2
import matplotlib.pyplot as plt

from common import (
   ROOT, LAB, DATA, RESULTS, FIGURES, PATTERN_SIZE, SQUARE_SIZE, SUBPIX_CRITERIA,
   imread, rgb, create_object_points, compute_reprojection_errors, save_csv
)

print("=== ЧастинаD: Дослідженнязбіжності, виправленнядисторсіїтаекспорт===")

# Завантаження24 валіднихвідфільтрованих() кадрівбезвикидуangle_8
file_list = []
for grp in ["frontal", "angles", "corners"]:
   flist = sorted(glob.glob(os.path.join(DATA, grp, "*.jpg")))
   for f in flist:
      if "angle_8.jpg" not in f:
         file_list.append((grp, f))

objp = create_object_points(PATTERN_SIZE, SQUARE_SIZE)
clean_objpoints = []
clean_imgpoints = []
h, w = None, None

for grp, fpath in file_list:
   img = imread(fpath)
   if h is None:
      h, w = img.shape[:2]
   gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
   ret, corners = cv2.findChessboardCorners(gray, PATTERN_SIZE)
   corners_sp = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), SUBPIX_CRITERIA)
   clean_objpoints.append(objp)
   clean_imgpoints.append(corners_sp.reshape(-1, 2))

# Базовекалібруваннянавсіх24 чистихкадрах
ret_final, K_final, D_final, rvecs_final, tvecs_final = cv2.calibrateCamera(
   clean_objpoints, clean_imgpoints, (w, h), None, None
)
print(f"Підсумкове калібрування(24 кадри): RMSE = {ret_final:.4f} px")

# ============================================================================
# Крок12: Залежністьпохибкитаfx відкількостікадрівN
# ============================================================================
np.random.seed(42)
random.seed(42)

Ns = [5, 8, 10, 12, 15, 20]
trials = 5
dependence_rows = []

n_stats = {}

for N in Ns:
   err_list = []
   fx_list = []
   fy_list = []
   for t in range(trials):
      idx = np.random.choice(len(clean_objpoints), N, replace=False)
      sub_obj = [clean_objpoints[i] for i in idx]
      sub_img = [clean_imgpoints[i] for i in idx]
```

```
        ret, K, D, _, _ = cv2.calibrateCamera(sub_obj, sub_img, (w, h), None, None)
        err_list.append(ret)
        fx_list.append(K[0, 0])
        fy_list.append(K[1, 1])

    mean_err, std_err = np.mean(err_list), np.std(err_list)
    mean_fx, std_fx = np.mean(fx_list), np.std(fx_list)
    mean_fy, std_fy = np.mean(fy_list), np.std(fy_list)

    n_stats[N] = {
        "err_mean": mean_err, "err_std": std_err,
        "fx_mean": mean_fx, "fx_std": std_fx,
        "fy_mean": mean_fy, "fy_std": std_fy
    }
    dependence_rows.append([
        N, f"{mean_err:.4f}", f"{std_err:.4f}",
        f"{mean_fx:.2f}", f"{std_fx:.2f}",
        f"{mean_fy:.2f}", f"{std_fy:.2f}"
    ])
    print(f"N = {N:2d}: RMSE = {mean_err:.4f} +/- {std_err:.4f} px, fx = {mean_fx:.2f} +/- {std_fx:.2f}
        px")

save_csv(
    os.path.join(RESULTS, "d_n_frames_dependence.csv"),
    ["N", "RMSE_mean", "RMSE_std", "fx_mean", "fx_std", "fy_mean", "fy_std"],
    dependence_rows
)
print("Збережено d_n_frames_dependence.csv")

# ПобудоваграфікавиходунаплатоКрок( 12)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))

# Графік1: fx відN
ax1.errorbar(
    Ns, [n_stats[N]["fx_mean"] for N in Ns],
    yerr=[n_stats[N]["fx_std"] for N in Ns],
    fmt="o-", color="#2b5c8f", ecolor="#2b5c8f", elinewidth=1.5, capsize=4,
    label="Оцінка $f_x$ середнє( $\\pm$ СВ)"
)
ax1.axhline(K_final[0, 0], color="red", linestyle="--", alpha=0.8,
            label=f"Еталон (24 кадри, {K_final[0, 0]:.1f} пк)")
ax1.axvspan(12, 20, color="green", alpha=0.08, label="Зона плато($N \\geq 12$)")
ax1.set_xlabel("Кількість кадрівкалібрування$N$", fontsize=10)
ax1.set_ylabel("Фокусна відстань$f_x$, пікселі", fontsize=10)
ax1.set_title("Збіжність оцінки$f_x$ зізростанням$N$", fontsize=11)
ax1.legend(loc="lower right", fontsize=8, framealpha=0.9)
ax1.grid(True, linestyle=":", alpha=0.6)

# Графік2: ПохибкарепроєкціївідN
ax2.errorbar(
    Ns, [n_stats[N]["err_mean"] for N in Ns],
    yerr=[n_stats[N]["err_std"] for N in Ns],
    fmt="s-", color="#d95f02", ecolor="#d95f02", elinewidth=1.5, capsize=4,
    label="Похибка RMSE середнє( $\\pm$ СВ)"
)
ax2.axhline(ret_final, color="red", linestyle="--", alpha=0.8,
            label=f"Еталон ({ret_final:.3f} пк)")
ax2.axvspan(12, 20, color="green", alpha=0.08, label="Зона плато($N \\geq 12$)")
ax2.set_xlabel("Кількість кадрівкалібрування$N$", fontsize=10)
ax2.set_ylabel("Похибка репроекціїRMSE, пікселі", fontsize=10)
ax2.set_title("Стабілізація похибкирепроекціївід$N$", fontsize=11)
ax2.legend(loc="lower right", fontsize=8, framealpha=0.9)
ax2.grid(True, linestyle=":", alpha=0.6)

plt.tight_layout()
fig.savefig(os.path.join(FIGURES, "d_n_frames_convergence.png"), dpi=160)
plt.close(fig)
print("Збережено d_n_frames_convergence.png")

# ============================================================================
# Крок13: Усуненнядисторсії: cv2.undistort зalpha = 0 таalpha = 1
# ============================================================================
sample_img_path = os.path.join(DATA, "corners", "corner_7.jpg")
sample_img = imread(sample_img_path)

# Матрицідляalpha = 0 (crop безчорнихполів) таalpha = 1 збереження( всьогополязполями)
```

```
newcameramtx_0, roi_0 = cv2.getOptimalNewCameraMatrix(K_final, D_final, (w, h), 0, (w, h))
newcameramtx_1, roi_1 = cv2.getOptimalNewCameraMatrix(K_final, D_final, (w, h), 1, (w, h))

undist_0 = cv2.undistort(sample_img, K_final, D_final, None, newcameramtx_0)
undist_1 = cv2.undistort(sample_img, K_final, D_final, None, newcameramtx_1)

# Візуальнепорівняннятрьохрежимів
fig, axes = plt.subplots(1, 3, figsize=(14, 4.2))

axes[0].imshow(rgb(cv2.resize(sample_img, (800, 600))))
axes[0].set_title("Вихідне зображенняз( дисторсією)", fontsize=9)

axes[1].imshow(rgb(cv2.resize(undist_0, (800, 600))))
axes[1].set_title("Виправлене: $\\alpha = 0$ кадр( обрізано)", fontsize=9)

axes[2].imshow(rgb(cv2.resize(undist_1, (800, 600))))
axes[2].set_title("Виправлене: $\\alpha = 1$ з( чорнимиполями)", fontsize=9)

for ax in axes:
    ax.axis("off")

plt.tight_layout()
fig.savefig(os.path.join(FIGURES, "d_undistort_alpha.png"), dpi=160)
plt.close(fig)
print("Збережено d_undistort_alpha.png")

# ============================================================================
# Крок14: Візуальнаперевіркапрямолінійностінапериферійнійлінії
# ============================================================================
# Накадріangle_10 рядоккутів5 проходитьчерезпериферійнузонусильноїдисторсії
line_img_path = os.path.join(DATA, "angles", "angle_10.jpg")
line_img = imread(line_img_path)
gray_line = cv2.cvtColor(line_img, cv2.COLOR_BGR2GRAY)
ret_chk, corners_chk = cv2.findChessboardCorners(gray_line, PATTERN_SIZE)
corners_chk_sp = cv2.cornerSubPix(gray_line, corners_chk, (11, 11), (-1, -1), SUBPIX_CRITERIA)

# Вибираєморядоккутівпоблизукраюрядок( 5 з9 точок)
pts_orig = corners_chk_sp.reshape(6, 9, 2)[5] # 9 точокуздовжпрямолінійногокраюшаблона

# Координатиточокпісляусуненнядисторсії(P=K зберігаєтусамуматрицю)
pts_undist = cv2.undistortPoints(pts_orig.reshape(-1, 1, 2), K_final, D_final, P=K_final).reshape(9, 2)

# Функціярозрахункувідхиленнястріли( прогину/ сагіти) відідеальноїпрямоїміжкрайнімиточками
def compute_deflections(points):
    p_start, p_end = points[0], points[-1]
    vec = p_end - p_start
    length = np.hypot(vec[0], vec[1])
    normal = np.array([-vec[1], vec[0]]) / length
    deflections = [float(np.dot(points[i] - p_start, normal)) for i in range(len(points))]
    return deflections

deflections_orig = compute_deflections(pts_orig)
deflections_undist = compute_deflections(pts_undist)

max_dev_orig = np.max(np.abs(deflections_orig))
max_dev_undist = np.max(np.abs(deflections_undist))
print(f"Крок 14: МаксимальневикривленнялініїДОвиправлення: {max_dev_orig:.2f} px")
print(f"Крок 14: МаксимальневикривленнялініїПІСЛЯвиправлення: {max_dev_undist:.2f} px")

# Побудовапорівняльногорисунка: фрагменткадрузнакладеноюпрямоютаграфіквідхилень
fig, (ax_img1, ax_img2, ax_plot) = plt.subplots(1, 3, figsize=(14, 4.2))

# Малюванняпрямоїопорноїхординаоригінальномуфрагменті
img_line_orig = line_img.copy()
cv2.line(
    img_line_orig,
    tuple(pts_orig[0].astype(int)),
    tuple(pts_orig[-1].astype(int)),
    (0, 0, 255), 4, cv2.LINE_AA
)
for pt in pts_orig:
    cv2.circle(img_line_orig, tuple(pt.astype(int)), 8, (0, 255, 0), -1)

# Навиправленомузображенні
img_undist_full = cv2.undistort(line_img, K_final, D_final)
cv2.line(
```

```
    img_undist_full,
    tuple(pts_undist[0].astype(int)),
    tuple(pts_undist[-1].astype(int)),
    (0, 0, 255), 4, cv2.LINE_AA
)
for pt in pts_undist:
    cv2.circle(img_undist_full, tuple(pt.astype(int)), 8, (0, 255, 0), -1)

# Збільшенийфрагментобластілінії
x_min = int(min(pts_orig[:, 0].min(), pts_undist[:, 0].min()) - 100)
x_max = int(max(pts_orig[:, 0].max(), pts_undist[:, 0].max()) + 100)
y_min = int(min(pts_orig[:, 1].min(), pts_undist[:, 1].min()) - 120)
y_max = int(max(pts_orig[:, 1].max(), pts_undist[:, 1].max()) + 120)

x_min, x_max = max(0, x_min), min(w, x_max)
y_min, y_max = max(0, y_min), min(h, y_max)

crop_orig = img_line_orig[y_min:y_max, x_min:x_max]
crop_undist = img_undist_full[y_min:y_max, x_min:x_max]

ax_img1.imshow(rgb(crop_orig))
ax_img1.set_title(f"ДО виправленнявикривлення( {max_dev_orig:.2f} пк)", fontsize=9)
ax_img1.axis("off")

ax_img2.imshow(rgb(crop_undist))
ax_img2.set_title(f"ПІСЛЯ виправленнявикривлення( {max_dev_undist:.2f} пк)", fontsize=9)
ax_img2.axis("off")

# Графіквідхиленьвідпрямоїуздовж9 точок
node_indices = np.arange(1, 10)
ax_plot.plot(node_indices, deflections_orig, "ro--", label=f"До виправленнямакс( {max_dev_orig:.2f} пк)")
ax_plot.plot(node_indices, deflections_undist, "bs-", label=f"Після виправленнямакс( {max_dev_undist:.2f}
    пк)")
ax_plot.axhline(0, color="black", linestyle=":", linewidth=0.8)
ax_plot.set_xlabel("Номер вузлавздовжлінії", fontsize=9)
ax_plot.set_ylabel("Відхилення відідеальноїпрямої, пк", fontsize=9)
ax_plot.set_title("Профіль прямолінійностіхорди", fontsize=10)
ax_plot.legend(loc="upper right", fontsize=8, framealpha=0.9)
ax_plot.grid(True, linestyle=":", alpha=0.6)

plt.tight_layout()
fig.savefig(os.path.join(FIGURES, "d_straight_line_check.png"), dpi=160)
plt.close(fig)
print("Збережено d_straight_line_check.png")

# Збереженняданихперевіркипрямолінійності
straight_rows = [
    [i + 1, f"{deflections_orig[i]:.3f}", f"{deflections_undist[i]:.3f}"]
    for i in range(9)
]
save_csv(
    os.path.join(RESULTS, "d_straight_line_profile.csv"),
    ["Вузол", "Відхиленнядопк__", "Відхиленняпісляпк__"],
    straight_rows
)
print("Збережено d_straight_line_profile.csv")

# ============================================================================
# Крок15: Збереженняcamera_params.yml черезcv2.FileStorage
# ============================================================================
yaml_path = os.path.join(RESULTS, "camera_params.yml")
fs = cv2.FileStorage(yaml_path, cv2.FILE_STORAGE_WRITE)

fs.write("calibration_date", "2026-10-29")
fs.write("camera_model", "Google Pixel 5")
fs.write("image_width", int(w))
fs.write("image_height", int(h))
fs.write("num_calibration_frames", int(len(clean_objpoints)))
fs.write("reprojection_error_rmse", float(ret_final))
fs.write("square_size_mm", float(SQUARE_SIZE))
fs.write("pattern_size_cols", int(PATTERN_SIZE[0]))
fs.write("pattern_size_rows", int(PATTERN_SIZE[1]))
fs.write("camera_matrix", K_final)
fs.write("distortion_coefficients", D_final)

fs.release()
```

```
print(f"Збережено {yaml_path}")

# Такождублюємоукореневийкаталог lab02/camera_params.yml длялегкогодоступу
root_yml = os.path.join(LAB, "camera_params.yml")
fs2 = cv2.FileStorage(root_yml, cv2.FILE_STORAGE_WRITE)
fs2.write("calibration_date", "2026-10-29")
fs2.write("camera_model", "Google Pixel 5")
fs2.write("image_width", int(w))
fs2.write("image_height", int(h))
fs2.write("num_calibration_frames", int(len(clean_objpoints)))
fs2.write("reprojection_error_rmse", float(ret_final))
fs2.write("square_size_mm", float(SQUARE_SIZE))
fs2.write("camera_matrix", K_final)
fs2.write("distortion_coefficients", D_final)
fs2.release()
print(f"Збережено {root_yml}")

# Записпідсумковоготекстовогожурналу
log_path = os.path.join(RESULTS, "calibration_log.txt")
with open(log_path, "w", encoding="utf-8") as f:
    f.write("=== ЗВІТПРОКАЛІБРУВАННЯКАМЕРИЛАБОРАТОРНА( РОБОТА№2) ===\n")
    f.write(f"Дата виконання: 2026-10-29\n")
    f.write(f"Камера: Google Pixel 5\n")
    f.write(f"Фізична роздільназдатність: {w} x {h}\n")
    f.write(f"Шаблон: {PATTERN_SIZE[0]}x{PATTERN_SIZE[1]} внутрішніхкутів, клітина= {SQUARE_SIZE} мм\n")
    f.write(f"Всього кадрівунаборі: 25\n")
    f.write(f"Вилучено викидів: 1 (angle_8.jpg, похибка2.951 пк> 2x медіани2.767 пк)\n")
    f.write(f"Використано дляфінальноїмоделі: 24 кадри\n")
    f.write(f"Підсумкова похибкарепроєкції(RMSE): {ret_final:.4f} пікселів\n\n")
    f.write(f"Матриця камериK:\n{K_final}\n\n")
    f.write(f"Коефіцієнти дисторсіїD:\n{D_final.ravel()}\n\n")
    f.write(f"fx = {K_final[0,0]:.2f}, fy = {K_final[1,1]:.2f}, fx/fy = {K_final[0,0]/K_final[1,1]:.4f}\n")
    f.write(f"cx = {K_final[0,2]:.2f} відхилення( відцентру{abs(K_final[0,2]-w/2)/(w/2)*100:.2f}%)\n")
    f.write(f"cy = {K_final[1,2]:.2f} відхилення( відцентру{abs(K_final[1,2]-h/2)/(h/2)*100:.2f}%)\n")

print(f"Збережено {log_path}")
print("Частина D успішновиконана!\n")
```